# GGPals Networking Architecture - Technical Documentation

## Table of Contents


1. Testing Guide
2. Architecture Overview
3. Layer 1: GGNetworking (Transport Library)
4. Layer 2: GGEngine (Engine Integration)
5. Layer 3: m5_networking (Gameplay Scripts)
6. Data Flow & Lifecycle
7. Wire Protocol Specification
8. Threading Model & Synchronization
9. RPC System Deep Dive
10. Turn-Based Game Flow
11. Complete File Inventory
12. Inter-Layer Communication
13. GameClient & GameServer in the Engine Update Loop


---

## 1. Testing Guide

### 1.1 Build Configuration

To test Networking, first build the Visual Studio project in **ReleaseEditor** build configuration. This will optimise the speed of the program's execution and improve load times (makes your grading easier).

### 1.2 Running the Instances

Once done building, run **3 instances** of the ReleaseEditor project. Since grading will be done on one machine, this machine will act as a localhost server, and 2 clients.

### 1.3 Setting Up the Server

On one of the running instances of the engine, navigate to the **Scenes** tab on the Assets Manager, then open **NetworkGame/M5/M5NetworkServer**. This is the Server scene. There may be an administrator request which you will have to approve for the server to bind when you first enter play mode.

### 1.4 Setting Up the Clients

On the other two instances of the engine, navigate to the **Scenes** tab on the Assets Manager, then open **M5LobbySelectionScene**. These will be the game clients. On the game clients, ensure that the entity **"NetworkedGameClient"** Script Component's member **"serverLink"** is set to **"127.0.0.1"**. This is important as this dictates the server you will connect to (IP address of localhost is 127.0.0.1).

### 1.5 Connecting and Creating a Lobby

Now, start both clients by entering Play mode. You will notice in the console (make sure **Info** is checked) that the clients have both been connected, and that there are 0 lobbies at the moment. Now click the **"Create Room"** button on the screen and notice that on the client you clicked "Create Room", you are automatically brought into the lobby room itself.

On the other client, you will also notice that there is a lobby entry of the same name as the room that was created on the other client. To join the room, you must click on the right arrow on the bottom left of the UI, on the right of the book, to change your faction to **Chaos**. Now, pressing **Join** will bring you into the lobby.

### 1.6 Starting the Game

Once in the lobby, press **"Ready"** on both clients, then press **Play** on any client, to enter the game.

Order N Chaos is an asymmetrical turn-based simultaneous resolution tactics game, where your own actions and your opponent's are buffered until a resolution phase resolves them in order. Since you are grading on localhost, there are no opponents, so you will be playing against yourself, on two clients.

### 1.7 Deployment Phase

First, you're brought into the Deployment Phase. Click and drag your units on the left/right side of the screen from their pedestals onto the board. You may deploy them in blue or red tiles. You will notice that the lighting of the ability costs will activate correspondingly to the number of different colored tiles your units are occupying. Once done deploying all 3 units, you may end your turn.

### 1.8 Preparation Phase

Once both clients have deployed their units and ended their turns, Preparation Phase begins. During Preparation Phase, you have **90 seconds** to perform any of these actions, in any order:

- Click on a tile a unit is occupying to **move** that unit (once per turn, per unit)
- Click on an ability in the abilities tab to **use** that ability (if the cost permits)

Once you have performed all the moves you wish to make, once again press **"End Turn"** and the game will proceed to Resolution Phase after your opponent finishes moving (this is a cue for you to go onto the other client to move the units!).

### 1.9 Resolution Phase

During Resolution Phase, all actions resolve in this particular order:

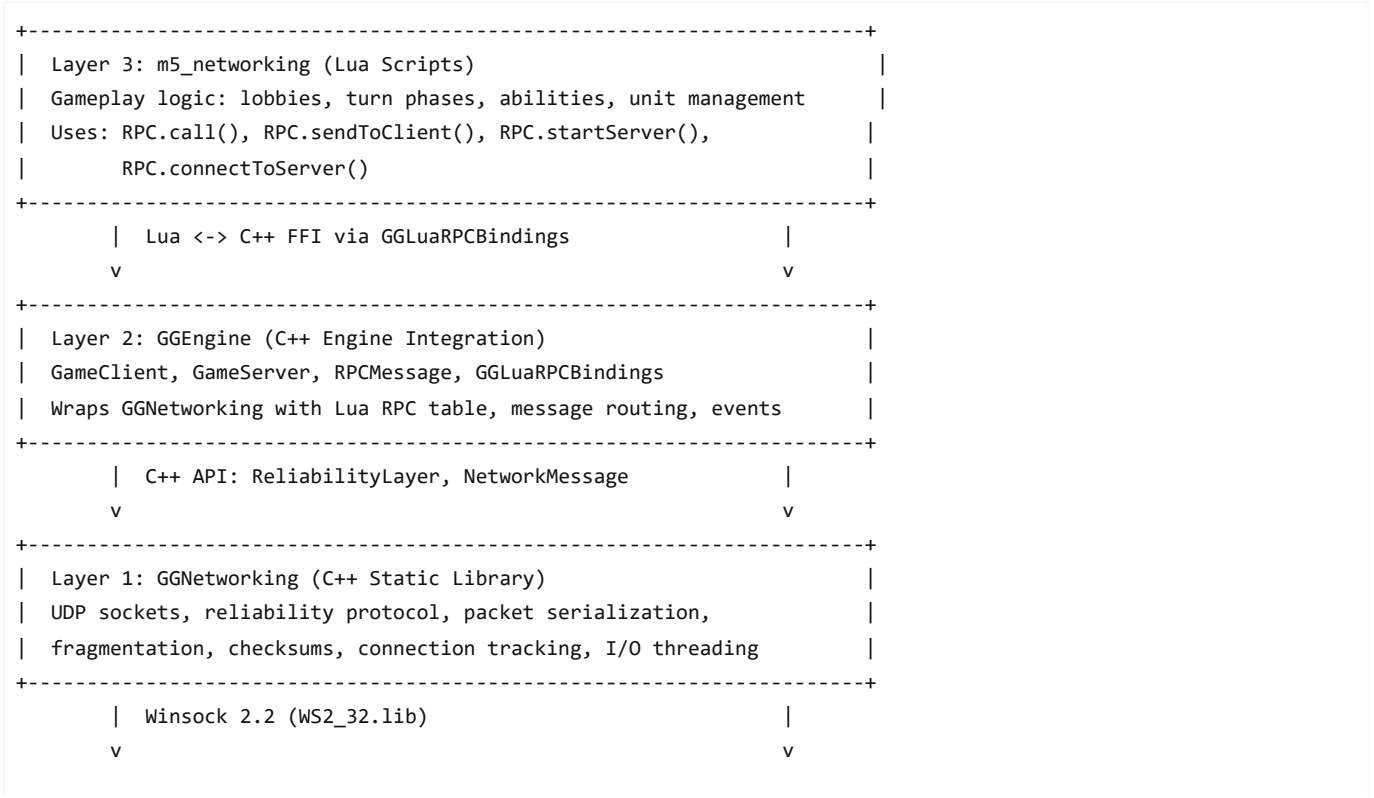
1. Order's movement and support abilities, in casting order
2. Order's attack abilities, in casting order
3. Chaos's movement and support abilities, in casting order
4. Chaos's attack abilities, in casting order

Congratulations! You have finished playing a turn in Order N Chaos.

---

## 2. Architecture Overview

The GGPals multiplayer networking system is organized into three distinct layers, each with clear responsibilities and interfaces:



```
+---------------------------------------------------------------------+
|  OS / Network Stack                                                 |
+---------------------------------------------------------------------+
```

### Design Principles

- **Separation of Concerns**: Transport (GGNetworking), engine integration (GGEngine), and gameplay (m5_networking) are fully decoupled.
- **Authoritative Server**: The server validates all gameplay actions (AP costs, unit positions, damage calculations) and broadcasts authoritative results.
- **Lua-First Gameplay**: Designers write gameplay networking logic entirely in Lua using a simple `RPC.call()` interface, without touching C++ code.
- **Custom Reliability over UDP**: A hand-rolled reliability layer provides sequencing, acknowledgments, retransmission, and fragmentation over raw UDP datagrams.

---

## 3. Layer 1: GGNetworking (Transport Library)

**Path**: `GGNetworking/include/` and `GGNetworking/src/` **Build Output**: Static library ( `.lib` ) **Author**: Tai Xiao Xuan

### 3.1 File Structure

| File | Role |
|---|---|
| `PlatformWindows.hpp/.cpp` | Winsock 2.2 initialization/shutdown |
| `UDPTypes.hpp` | Type aliases, constants, enums (PacketType, MessageType) |
| `UDPPacket.hpp/.cpp` | Transport-layer packet with header, checksum, serialization |
| `UDPMessage.hpp/.cpp` | Application-layer message with type-safe serialization |
| `UDP.hpp` | `ClientConnection` struct (per-client state) |
| `UDPReliabilityLayer.hpp/.cpp` | Core reliability layer with threading and connection management |

### 3.2 Type System

Defined in `UDPTypes.hpp` :

```
using PlayerID   = uint32_t;   // 0 = invalid/server, 1-255 = players
using SessionID  = uint64_t;   // Session token (reserved for reconnection)
using SequenceNum = uint32_t;  // Packet ordering
```

### 3.3 Configuration Constants

| Constant | Value | Purpose |
|---|---|---|
| `DEFAULT_SERVER_PORT` | 9081 | Fixed server port |
| `MAX_PLAYERS` | 256 | Maximum concurrent connections |
| `MAX_PACKET_SIZE` | 1400 bytes | Single datagram limit (fits in typical MTU) |
| `MAX_MESSAGE_SIZE` | 16384 bytes (16 KB) | Maximum application message (fragmented if > packet size) |
| `CONNECTION_TIMEOUT_MS` | 30000 ms | Connection considered dead after 30s |
| `PACKET_TIMEOUT_MS` | 1000 ms | Retransmission trigger threshold |

| | | |
|---|---|---|
| MAX_RETRANSMISSIONS | 5 | Packet dropped after 5 retries |
| CLIENT_PING_INTERVAL_MS | 2000 ms | Client heartbeat frequency |
| HEARTBEAT_TIMEOUT_US | 10,000,000 us (10s) | Server-side timeout per client |
| MAX_FRAGMENT_PAYLOAD | ~1383 bytes | Per-fragment data capacity |
| FRAGMENT_TIMEOUT_MS | 10000 ms | Reassembly timeout for incomplete fragments |

### 3.4 Packet Types (Transport Layer)

Defined as `PacketType` enum ( `UDPTypes.hpp:88` ):

| Type | Value | Description |
|---|---|---|
| DATA | 0 | Regular data packet carrying a `NetworkMessage` payload |
| ACK | 1 | Acknowledgment (echoes the sequence number being ACKed) |
| HEARTBEAT | 2 | Keep-alive signal |
| CONNECT | 3 | Connection initiation (reserved, actual handshake uses DATA + CLIENT_CONNECT_REQUEST) |
| DISCONNECT | 4 | Clean disconnection |
| FRAGMENT | 5 | Fragment of a large message |

### 3.5 Message Types (Application Layer)

Defined as `MessageType` enum ( `UDPTypes.hpp:108` ):

| Category | Type | Value | Direction |
|---|---|---|---|
| Connection | CLIENT_CONNECT_REQUEST | 100 | Client -> Server |
| Connection | SERVER_CONNECT_RESPONSE | 101 | Server -> Client |
| Connection | CLIENT_DISCONNECT_REQUEST | 102 | Client -> Server |
| Connection | CLIENT_DISCONNECT_RESPONSE | 103 | Server -> Client |
| Connection | CLIENT_PING | 104 | Client -> Server |
| Connection | SERVER_PONG | 105 | Server -> Client |
| RPC | RPC_CALL | 199 | Bidirectional |
| Game State | SERVER_GAME_STATE | 200 | Server -> Client |
| Turn | SERVER_TURN_START | 201 | Server -> Client |
| Turn | CLIENT_TURN_ACTION | 202 | Client -> Server |
| Turn | SERVER_TURN_RESULT | 203 | Server -> Client |
| Player | SERVER_PLAYER_JOINED | 300 | Server -> Client |
| Player | SERVER_PLAYER_LEFT | 301 | Server -> Client |
| Game Flow | SERVER_GAME_START | 400 | Server -> Client |

| | | | |
|---|---|---|---|
| Game Flow | `SERVER_GAME_END` | 401 | Server -> Client |
| Error | `SERVER_ERROR` | 500 | Server -> Client |

### 3.6 PacketHeader Structure

`UDPPacket.hpp:50` - 9 bytes, `#pragma pack(1)` :

```
Offset  Size  Field
0       4     sequenceNum    (uint32_t)
4       2     payloadSize    (uint16_t)
6       2     checksum       (uint16_t, RFC 1071 Internet Checksum)
8       1     packetType     (PacketType enum, uint8_t)
```

The checksum covers the header only (not payload), computed using the RFC 1071 algorithm:

1. Zero the checksum field
2. Sum bytes 0-5 as 16-bit words (sequenceNum + payloadSize)
3. Add byte 8 (packetType)
4. Fold carries into 16-bit result
5. Store one's complement (~sum)

Validation re-computes the checksum and compares ( `UDPPacket.cpp:254` ).

### 3.7 MessageHeader Structure

`UDPMessage.hpp:115` - 10 bytes, `#pragma pack(1)` :

```
Offset  Size  Field
0       2     messageType    (MessageType enum, uint16_t)
2       4     payloadSize    (uint32_t)
6       4     messageID      (uint32_t, auto-assigned from static counter)
```

### 3.8 FragmentHeader Structure

`UDPTypes.hpp:69` - 8 bytes, `#pragma pack(1)` :

```
Offset  Size  Field
0       4     messageID       (uint32_t, shared across all fragments of same message)
4       2     fragmentIndex   (uint16_t, 0-based)
6       2     fragmentTotal   (uint16_t, total fragment count)
```

### 3.9 NetworkMessage Serialization

`NetworkMessage` ( `UDPMessage.hpp:140` ) provides type-safe binary serialization constrained by the `IsNetworkSerializable` concept:

```
template<typename T>
concept IsNetworkSerializable = requires {
    requires std::is_trivially_copyable_v<T>;
    requires std::is_standard_layout_v<T>;
    requires !std::is_pointer_v<T>;
    requires !std::is_reference_v<T>;
};
```

This enables zero-copy `memcpy` -based serialization for POD types. The `WriteValue<T>()` method appends `sizeof(T)` bytes to the payload; `ReadValue<T>()` reads sequentially with bounds checking.

Raw byte access is also provided via `WriteRawBytes()` for composite serialization (used by the RPC system for variable-length argument data).

### 3.10 ClientConnection (Per-Client State)

`UDP.hpp:61` - One instance per connected player, stored in `ReliabilityLayer::connections`:

| Field | Type | Purpose |
|---|---|---|
| `playerAddress` | `sockaddr_in` | Target address for `sendto()` |
| `playerID` | `PlayerID` | Unique identifier (1-255) |
| `sessionID` | `SessionID` | Session token (future reconnection support) |
| `nextSendSequence` | `SequenceNum` | Next outgoing sequence number (starts at 1) |
| `expectedReceiveSequence` | `SequenceNum` | Next expected incoming sequence |
| `pendingPackets` | `map<SequenceNum, Packet>` | Unacknowledged outgoing packets awaiting ACK |
| `receivedPackets` | `map<SequenceNum, bool>` | Received sequence numbers for duplicate detection |
| `lastHeartbeat` | `time_point` | Last heartbeat/ping received |
| `lastActivity` | `time_point` | Last packet of any type received |
| `isConnected` | `bool` | Connection established flag |
| `timeout` | `bool` | Marked for removal by timeout check |
| `fragmentBuffer` | `map<uint32_t, vector<vector<uint8_t>>>` | In-progress fragment reassembly |

### 3.11 ReliabilityLayer

`UDPReliabilityLayer.hpp:59` - The core networking engine providing TCP-like reliability over UDP.

**Key Members:**

- `connections` ( `map<PlayerID, unique_ptr<ClientConnection>>` ) - All active connections
- `incomingMessages` ( `queue<pair<NetworkMessage, PlayerID>>` ) - Messages ready for application consumption
- `coldMessages` - Double-buffered message queue (swapped during client Update for lock-free reads)
- `fragmentAssembly` ( `map<uint32_t, FragmentAssembly>` ) - In-progress fragment reassemblies
- `availablePlayerIDs` ( `queue<PlayerID>` ) - Reusable ID pool (1 to MAX_PLAYERS-1)
- `socketIOThread` - Dedicated I/O thread
- `connectionsMutex` / `messagesMutex` - `shared_mutex` for thread-safe access

**Lifecycle:**

1. Constructed with a pre-bound, non-blocking UDP socket
2. `Start()` populates the player ID pool and spawns the I/O thread
3. I/O thread runs `ReceiveThreadProc()` continuously
4. `Stop()` signals thread exit, joins, and closes the socket

**I/O Thread Main Loop** ( `UDPReliabilityLayer.cpp:191` ):

```
while (shouldRun):
    if (!ReceivePacket()):
        sleep(1ms)  // Avoid busy-wait

    // Retransmission scan
```

```
for each connection:
    for each pendingPacket:
        if !acknowledged && elapsed > PACKET_TIMEOUT_MS:
            if retransmissionCount >= MAX_RETRANSMISSIONS:
                erase packet (give up)
            else:
                retransmit, increment counter
```

**Packet Reception Pipeline** ( `ReceivePacket()` , `UDPReliabilityLayer.cpp:394` ):

1. `recvfrom()` on non-blocking socket
2. Validate minimum size (>= 9 bytes for PacketHeader)
3. `Packet::Deserialize()` - validate header, checksum, extract payload
4. Dispatch by `PacketType` :
   - **ACK**: Find pending packet by sequence number, mark acknowledged, calculate RTT (exponential moving average: `0.9 * old + 0.1 * new` )
   - **HEARTBEAT**: Update `lastActivity` timestamp, send ACK back
   - **DATA**: Send ACK immediately, deserialize `NetworkMessage` from payload, handle `CLIENT_CONNECT_REQUEST` specially (allocate PlayerID), enqueue to `incomingMessages`
   - **FRAGMENT**: Send ACK, extract `FragmentHeader` , store fragment in `FragmentAssembly` , reassemble when all fragments received, clean up stale assemblies (> `FRAGMENT_TIMEOUT_MS` )

**Statistics Tracking** ( `ReliabilityStats` ):

- `packetsSent` , `packetsReceived` , `packetsRetransmitted` , `packetsLost` , `duplicatesReceived`
- `averageRoundTripTime` (exponential moving average)

### 3.12 Fragmentation Algorithm

When a serialized `NetworkMessage` exceeds `MAX_PACKET_SIZE` (1400 bytes including headers), it is split into multiple FRAGMENT packets:

**Sending** (in `GameClient::SendToServer` and `GameServer::SendFragmented` ):

1. Serialize the `NetworkMessage` to bytes
2. Calculate fragment count: `ceil(serialized.size / MAX_FRAGMENT_PAYLOAD)`
3. For each fragment index `i` :
   - Create a `FragmentHeader` with shared `messageID` , `fragmentIndex = i` , `fragmentTotal`
   - Slice `[i * MAX_FRAGMENT_PAYLOAD, min((i+1) * MAX_FRAGMENT_PAYLOAD, end)]`
   - Pack into a FRAGMENT `Packet` : payload = FragmentHeader + data slice
   - Send via `sendto()`

**Receiving** (in `ReliabilityLayer::ReceivePacket` ):

1. Validate fragment payload >= `FRAGMENT_HEADER_SIZE` (8 bytes)
2. Extract `FragmentHeader` , validate `fragmentIndex < fragmentTotal`
3. Look up or create `FragmentAssembly` entry by `messageID`
4. Store fragment data (skip duplicates)
5. When `receivedCount == fragmentTotal` :
   - Concatenate all fragment data in order
   - Deserialize as `NetworkMessage`
   - Enqueue to `incomingMessages`
   - Clean up assembly entry
6. Periodically purge stale assemblies (> 10 second timeout)

### 3.13 Platform Layer

`PlatformWindows.hpp/.cpp` - Winsock 2.2 abstraction:

- `NetworkUtils::InitWindowsNetworking()` - Calls `WSAStartup(2,2)`, validates version
- `NetworkUtils::ShutdownWindowsNetworking()` - Calls `WSACleanup()`

The namespace design (`NetworkUtils`) allows future cross-platform support with POSIX socket implementations.

---

## 4. Layer 2: GGEngine (Engine Integration)

**Path**: `GGEngine/include/` and `GGEngine/src/Networking/`, `GGEngine/src/ScriptingSystem/Networking/`

This layer bridges the generic GGNetworking library with the GGEngine runtime, adding:

- Singleton accessors for client/server instances
- Lua RPC bindings for script-driven networking
- Engine event bus integration (play mode lifecycle)
- Time synchronization between client and server
- Message routing to Lua function calls

### 4.1 File Structure

| File | Role |
|---|---|
| `GameClient.hpp/.cpp` | Client-side network manager |
| `GameServer.hpp/.cpp` | Server-side network manager |
| `RPCMessage.hpp/.cpp` | Lua RPC serialization/deserialization |
| `GGLuaRPCBindings.hpp/.cpp` | Lua-to-C++ binding registration |

### 4.2 GameClient

`GameClient.hpp:58` - Singleton via `GetGameClient()`.

**Initialization Flow:**

1. `Init()` - Creates UDP socket (`AF_INET, SOCK_DGRAM, IPPROTO_UDP`), sets non-blocking mode via `ioctlsocket(FIONBIO)`, binds to OS-assigned port, creates `ReliabilityLayer`
2. `ConnectToServer(host, port)` - Resolves hostname via `getaddrinfo()`, starts ReliabilityLayer I/O thread, sends `CLIENT_CONNECT_REQUEST` with client timestamp, marks `isRunning = true`

**Connection Handshake:**

```
Client                          Server
  |                               |
  |--- CLIENT_CONNECT_REQUEST ---->|  (contains client timestamp)
  |                               |  Server assigns PlayerID,
  |                               |  creates ClientConnection
  |<-- SERVER_CONNECT_RESPONSE ----|  (contains PlayerID, server time, status)
  |                               |
  |  Client stores PlayerID,      |
  |  calculates time offset,      |
  |  fires Lua callback:          |
  |  ClientRPC_OnConnectedToServer |
  |                               |
  |--- CLIENT_PING --------------->|  (immediate heartbeat)
  |<-- SERVER_PONG ----------------|
  |                               |
  |  (periodic pings every 2s)    |
```

**Update Loop** ( `GameClient::Update()` , `GameClient.cpp:238` ):

1. Send periodic `CLIENT_PING` (every `CLIENT_PING_INTERVAL_MS` )
2. Swap incoming message queue (double-buffer pattern for lock-free reads):

```
std::swap(reliabilityLayer->incomingMessages, reliabilityLayer->coldMessages);
```

3. Process all messages in `coldMessages` :
   - `SERVER_CONNECT_RESPONSE` : Extract PlayerID, calculate time offset, call `ClientRPC_OnConnectedToServer(playerID)` in Lua
   - `CLIENT_DISCONNECT_RESPONSE` : Reset state, call `ClientRPC_OnDisconnectedFromServer()` in Lua
   - `SERVER_PONG` : Calculate RTT ( `currentTime - lastPingSentTime` ), update `clientServerOffset` using half-RTT correction
   - `RPC_CALL` : Deserialize via `LuaRPCDeserializer` and execute in Lua

**Time Synchronization:**

- `clientServerOffset` is computed during handshake: `serverTime - clientTime`
- Refined on each PONG: `serverTimeReceived - lastPingSentTime - halfRTT`
- `GetGameServerTime()` returns `GetGameClientTime() + clientServerOffset`

**Message Sending** ( `SendToServer()` , `GameClient.cpp:386` ):

- Serializes `NetworkMessage` to bytes
- If total wire size (PacketHeader + serialized message) <= 1400 bytes: single DATA packet via `sendto()`
- If larger: fragments using the fragmentation algorithm (see Section 2.12)

**Lifecycle Integration:**

- Constructor subscribes to `GlobalEventBus::onStopPlayMode` -> calls `StopClient()`
- `StopClient()` sends disconnect, stops reliability layer, closes socket

### 4.3 GameServer

`GameServer.hpp:48` - Singleton via `GetGameServer()` .

**Initialization Flow:**

1. `Init()` - Creates UDP socket, sets non-blocking mode
2. `StartServer(port)` - Binds to port (default 9081), creates and starts ReliabilityLayer

**Update Loop** ( `GameServer::Update()` , `GameServer.cpp:209` ):

1. Process all `incomingMessages` (no double-buffering, reads directly under lock):
   - `CLIENT_CONNECT_REQUEST` : Update connection timestamps, call `ServerRPC_OnClientConnected(playerID)` in Lua, respond with `SERVER_CONNECT_RESPONSE` containing assigned PlayerID and server time
   - `CLIENT_DISCONNECT_REQUEST` : Remove connection, call `ServerRPC_OnClientDisconnected(playerID)` in Lua
   - `CLIENT_PING` : Update heartbeat timestamp, respond with `SERVER_PONG`
   - `RPC_CALL` : Deserialize and execute via `LuaRPCDeserializer`
   - `CLIENT_TURN_ACTION` : Broadcast to all other clients (excluding sender)
2. Heartbeat timeout check: For each connection, if `lastHeartbeat` exceeds `HEARTBEAT_TIMEOUT_US` (10 seconds), mark as timed out and remove

**Broadcasting** ( `BroadcastMessage()` , `GameServer.cpp:355` ):

- Serializes message once, sends to all connections (optionally excluding one player)
- Handles fragmentation per-client for large messages

**Targeted Sending** ( `SendToClient()` , `GameServer.cpp:428` ):

- Looks up connection by PlayerID, sends directly
- Supports fragmentation for large messages

### 4.4 RPCMessage System

`RPCMessage.hpp/.cpp` - Serializes Lua function calls into `NetworkMessage` payloads.

#### RPCMessageHeader

`RPCMessage.hpp:51` - 65 bytes, `#pragma pack(1)`:

```
Offset  Size  Field
0       1     functionNameLength   (uint8_t, max 62)
1       63    functionName         (char[63], null-terminated)
64      1     argCount             (uint8_t, max 255)
```

#### Argument Type Tags

```
enum class ArgType : uint8_t {
    ARG_NIL     = 0,   // No payload
    ARG_BOOLEAN = 1,   // 1 byte (0 or 1)
    ARG_INTEGER = 2,   // 4 bytes (int32_t)
    ARG_STRING  = 3,   // 1 byte length + N bytes data (max 255)
    ARG_NUMBER  = 4,   // 8 bytes (double)
    ARG_ENTITY  = 5,   // 8 bytes (uint64_t packed entity handle)
    ARG_ARRAY   = 6,   // 2 bytes count + N recursive elements
    ARG_TABLE   = 7    // 2 bytes count + N recursive key-value pairs
};
```

#### LuaRPCSerializer

`RPCMessage.hpp:75` - Builds RPC messages from Lua values:

1. Constructor takes function name (max 62 chars, truncated if longer)
2. `AddArg()` overloads for bool, double, int32_t, string, entity
3. `WriteTypeByte()` / `WriteRawByte()` / `WriteRawData()` for composite types
4. `Serialize(NetworkMessage&)` writes:
   - Sets message type to `RPC_CALL` (199)
   - Writes `RPCMessageHeader` as POD value
   - Appends raw argument data bytes

#### LuaRPCDeserializer

`RPCMessage.cpp:151` - Executes RPC calls in Lua:

1. Read `RPCMessageHeader` from message payload
2. Look up function by name in Lua global table ( `lua_getglobal` )
3. For each argument ( `argCount` times):
   - Read `ArgType` tag byte
   - Read argument value based on type
   - Push onto Lua stack
   - Arrays/tables are recursively deserialized
4. Call function via `lua_pcall(L, argCount, 0, 0)`

#### Wire Format of an RPC_CALL Message:

```
[MessageHeader: 10 bytes]
  [messageType = 199 (RPC_CALL)]
```

```
  [payloadSize]
  [messageID]
[RPCMessageHeader: 65 bytes]
  [functionNameLength: 1 byte]
  [functionName: 63 bytes]
  [argCount: 1 byte]
[Argument Data: variable]
  [ArgType: 1 byte] [value: variable] ...
  [ArgType: 1 byte] [value: variable] ...
```

### 4.5 GGLuaRPCBindings

`GGLuaRPCBindings.hpp/.cpp` - Registers the `RPC` global table in Lua with four functions:

#### RPC.call(functionName, arg1, arg2, ...)

`GGLuaRPCBindings.cpp:76` - Universal RPC dispatcher:

1. Reads function name from Lua stack position 1
2. Serializes all arguments (positions 2+) via `AddLuaValueToRPC()`
3. Routes automatically:
   - If GameClient is running: sends to server via `SendToServer()`
   - If GameServer is running: broadcasts to all clients via `BroadcastMessage()`
   - Otherwise: raises Lua error

#### RPC.sendToClient(playerID, functionName, arg1, arg2, ...)

`GGLuaRPCBindings.cpp:401` - Server-only targeted RPC:

1. Reads target PlayerID from position 1
2. Reads function name from position 2
3. Serializes remaining arguments
4. Sends to specific client via `GameServer::SendToClient()`

#### RPC.startServer(port?)

`GGLuaRPCBindings.cpp:305` - Initializes and starts the game server:

1. Optional port parameter (default 9081)
2. Calls `GameServer::Init()` then `GameServer::StartServer(port)`
3. Returns boolean success to Lua

#### RPC.connectToServer(address?, port?)

`GGLuaRPCBindings.cpp:349` - Connects client to server:

1. Optional address (default "eisenfaust.ddns.net") and port (default 9081)
2. Calls `GameClient::Init()` then `GameClient::ConnectToServer()`
3. Returns boolean success to Lua

#### Lua Value Serialization

`WriteLuaValueToRPC()` ( `GGLuaRPCBindings.cpp:139` ) recursively serializes Lua values:

- **Booleans**: `[ARG_BOOLEAN][0 or 1]`
- **Numbers**: Auto-detects integer vs. float. Integers ( `floor(n) == n` and within int32 range) serialize as `[ARG_INTEGER][4 bytes]` ; otherwise `[ARG_NUMBER][8 bytes double]`
- **Strings**: `[ARG_STRING][length byte][string bytes]` (max 255 chars)
- **Light userdata**: `[ARG_ENTITY][8 bytes uint64]`
- **Tables**: Classified as array or table:
  - **Array** (all keys are sequential integers 1..N): `[ARG_ARRAY][2 bytes count][elements...]`

- **Table** (mixed/string keys): `[ARG_TABLE][2 bytes count][key, value, key, value...]`

- **Nil**: `[ARG_NIL]` (no additional bytes)
- Max nesting depth: 8 levels

---

## 5. Layer 3: m5_networking (Gameplay Scripts)

**Path**: `Assets/Scripts/m5_networking/`

This layer implements the actual game networking logic in Lua, using the RPC interface exposed by Layer 2. It follows a client-server architecture for a turn-based tactics game (two-player, grid-based, with units and abilities).

### 5.1 Directory Structure

```
m5_networking/
  NetworkRPCHandler.lua         # Action queue handler (stub)
  NetworkTestButton.lua         # Dev/test UI for starting server/client
  System/
    ConnectionCallbacks.lua     # Engine-to-Lua connection event handlers
  Server/
    NetworkedGameServer.lua     # Server-side game logic (lobbies, resolution)
  Client/
    NetworkedGameClient.lua     # Client-side connection + lobby join
    ClientGameManager.lua       # Client game state manager
    Grid/
      ClientGrid.lua            # Client grid state
      ClientGridLink.lua        # Grid visual linking
    TurnState/
      ClientTurnPhases.lua      # Phase state machine
      ClientDeploymentPhase.lua # Unit placement phase
      ClientPreparationPhase.lua# Action planning phase
      ClientResolutionPhase.lua # Animation/replay phase
    Unit/
      ClientUnit.lua            # Unit representation
      StateMachine/             # Unit animation states
    Abilities/
      ClientAbility.lua         # Base ability class
      ClientAbilities.lua       # Ability collection
      Dragoon/, Eye/, Imp/, Oracle/, Tirion/, Vizom/  # Per-character abilities
```

### 5.2 Connection Callbacks

`System/ConnectionCallbacks.lua` - Bridges C++ engine events to Lua local events:

| C++ Callback | Lua Function | Emits Local Ever |
|---|---|---|
| `CallLuaFunctionAdvanced("ServerRPC_OnClientConnected", playerID)` | `ServerRPC_OnClientConnected(clientID)` | `OnClientConnected` |
| `CallLuaFunctionAdvanced("ClientRPC_OnConnectedToServer", playerID)` | `ClientRPC_OnConnectedToServer(assignedClientID)` | `OnConnectedToServer` |
| `CallLuaFunctionAdvanced("ServerRPC_OnClientDisconnected", playerID)` | `ServerRPC_OnClientDisconnected(clientID)` | `OnClientDisconnecte` |
| `CallLuaFunctionAdvanced("ClientRPC_OnDisconnectedFromServer")` | `ClientRPC_OnDisconnectedFromServer()` | `OnDisconnectedFromS` |

### 5.3 NetworkedGameClient

`Client/NetworkedGameClient.lua` - Client-side connection manager:

**Serializable Properties:**

- `serverLink` - Server address (default "127.0.0.1")
- `lobbyName` - Target lobby name (default "TestLobby")
- `password` - Lobby password

**Lifecycle:**

1. `Init()` : Subscribes to `OnConnectedToServer` event. On connection, sends `ServerRPC_CreateOrJoinLobby` RPC with client ID, lobby name, password, and chosen faction
2. `Start()` : Calls `RPC.connectToServer(self.serverLink, 9081)`

**RPC Handlers (global functions):**

- `ClientRPC_OnReturnDeployUnit(unitType, gridPosX, gridPosY)` - Acknowledges unit deployment
- `ClientRPC_OnReturnEndDeploymentPhase(success)` - Deployment validation result
- `ClientRPC_OnEndDeploymentPhase(oracleX, oracleY, ..., vizomX, vizomY)` - All units deployed, syncs positions, transitions to preparation phase
- `ClientRPC_OnReturnLobbyJoined(lobbyName)` - Lobby join confirmation
- `ClientRPC_OnResolutionData(data)` - Receives server-calculated turn results, transitions to resolution phase

### 5.4 NetworkedGameServer

`Server/NetworkedGameServer.lua` - Authoritative server-side game logic.

**State:**

- `connectedClients` - Table of connected client IDs
- `lobbyList` - Table of active lobbies (keyed by name)

**Lobby Structure:**

```
{
    name = "LobbyName",
    password = "...",
    clients = { clientID1, clientID2 },
    gameData = {
        units = { {}, {}, {}, {}, {}, {} },  -- 6 units (3 Order + 3 Chaos)
        order = { client = clientID, ready = false, moveSupportQueue = {}, attackQueue = {} },
        chaos = { client = clientID, ready = false, moveSupportQueue = {}, attackQueue = {} },
        phase = "deployment" | "preparation" | "gameover",
        turnNumber = 1,
        turnHistory = {},
        tileHealth = { order = {}, chaos = {} },
        traps = { order = {}, chaos = {} },
        playerHealth = { order = MAX_HP, chaos = MAX_HP }
    }
}
```

**RPC Handlers (global functions called by clients via RPC.call):**

| Function | Purpose |
| --- | --- |
| `ServerRPC_CreateOrJoinLobby(clientID, lobbyName, password, faction)` | Creates or joins a lobby, assigns faction |

| | |
|---|---|
| `ServerRPC_OnDeployUnit(clientID, unitType, gridPosX, gridPosY)` | Validates and records unit placement |
| `ServerRPC_OnClientEndDeploymentPhase(clientID)` | Validates all units deployed; when both players ready, broadcasts positions and transitions to preparation phase |
| `ServerRPC_OnSubmitActions(clientID, moveSupportQueue, attackQueue)` | Validates AP cost, stores queued actions; when both players ready, triggers resolution |

**Server-Side Ability Data:**

The server maintains its own ability definitions ( `SERVER_ABILITY_DATA` ) keyed by ability ID, containing:

- `cost` - AP cost
- `targetsOpponent` - Whether ability targets opponent grid
- `isSupport` / `isTrap` / `isShield` - Ability classification
- `unitDamage` / `tileDamage` / `healAmount` - Effect values
- `GetTargetTiles(targetX, targetY)` - Returns list of affected tile indices

### 5.5 Resolution Algorithm (Server-Authoritative)

`NetworkedGameServer:BroadcastResolution(lobby)` ( `NetworkedGameServer.lua:572` ):

1. **Arm Existing Traps**: All unarmed traps from previous turns become armed
2. **Snapshot Positions**: Record current unit positions as `syncPositions`
3. **Consolidate Actions** in execution order:
   - Order player's move/support queue
   - Chaos player's move/support queue
   - Order player's attack queue
   - Chaos player's attack queue
4. **Pre-Calculate Results** ( `PreCalculateResults` ):
   - For each action, compute:
     - **Movements**: Update unit position, check path for armed traps (via C++ pathfinding or Manhattan fallback)
     - **Abilities**: Determine target tiles, check for unit hits (with shield checking), calculate tile damage, apply healing, place traps
   - Track shields: Shield abilities add entries to `shieldedUnits` , blocking the next damage instance
5. **Apply to Server State**: Update tile health, player health, remove triggered traps, apply position changes
6. **Broadcast**: Send `ClientRPC_OnResolutionData` to all clients with:
   - `turn` - Current turn number
   - `syncPositions` - Unit positions before this turn's movements
   - `queue` - Array of pre-calculated action results
   - `finalHealth` - Post-resolution health for both players
   - `winner` - Winner flag (0 = no winner, 1 = Order, 2 = Chaos, 3 = tie)
7. **Reset Queues**: Clear action queues, advance turn number (or set phase to "gameover")

### 5.6 Client Turn State Machine

`Client/TurnState/ClientTurnPhases.lua` - Simple state machine:

```
DeploymentPhase --> PreparationPhase --> ResolutionPhase --> PreparationPhase --> ...
                                                        |
                                                        +--> Game Over
```

**Deployment Phase**

- Players place 3 units each on their grid
- Each placement sends `ServerRPC_OnDeployUnit` via RPC

- "End deployment" sends `ServerRPC_OnClientEndDeploymentPhase`
- Server waits for both players, then broadcasts all positions

**Preparation Phase ( `ClientPreparationPhase.lua` )**

- Timed phase ( `PREP_PHASE_TIMER` seconds)
- Player selects abilities (deducting AP) and moves units
- Actions are queued locally in `localMoveSupportQueue` and `localAttackQueue`
- On submit (button press or timer expiry), sends `ServerRPC_OnSubmitActions` via RPC
- Sets `awaitingServer = true` to block further input

**Resolution Phase ( `ClientResolutionPhase.lua` )**

- Receives pre-calculated results from server
- Replays actions sequentially with visual animations:
  - `RESOLVE_SETTLING` -> `RESOLVE_NEXT_ACTION` -> `RESOLVE_MOVING` or `RESOLVE_ABILITY` -> `RESOLVE_DELAY` -> next action
- For movements: Emits `ReplayMoveUnit` event, waits for `OnReplayMoveComplete`
- For abilities: Triggers ability animation, waits for `OnAbilityTriggerComplete`
- Applies server-authoritative damage, healing, tile effects, and trap triggers
- Syncs final health from server data
- Checks for winner, transitions to next preparation phase or game over

---

# 6. Data Flow & Lifecycle

## 6.1 Full Connection Lifecycle

```
[Lua] RPC.connectToServer("host", 9081)
  -> [C++] GameClient::Init()           // Create socket, bind, create ReliabilityLayer
  -> [C++] GameClient::ConnectToServer() // Resolve host, start I/O thread
     -> [Net] CLIENT_CONNECT_REQUEST    // UDP datagram to server
        -> [C++ Server] ReliabilityLayer assigns PlayerID
        -> [C++ Server] GameServer::Update() processes message
           -> [Lua] ServerRPC_OnClientConnected(playerID)
        -> [Net] SERVER_CONNECT_RESPONSE  // Contains PlayerID, server time
     -> [C++ Client] GameClient::Update() processes response
        -> [Lua] ClientRPC_OnConnectedToServer(playerID)
           -> [Lua] RPC.call("ServerRPC_CreateOrJoinLobby", clientID, lobbyName, password, faction)
              -> ... (gameplay begins)
```

## 6.2 RPC Call Flow (Client to Server)

```
[Lua Client] RPC.call("ServerRPC_OnSubmitActions", clientID, moveSupportQueue, attackQueue)
  |
  v
[C++ GGLuaRPCBindings] lua_RPC_Call(L)
  1. Extract function name "ServerRPC_OnSubmitActions"
  2. Serialize args (int, table, table) via WriteLuaValueToRPC
     - Tables serialized recursively with ARG_ARRAY/ARG_TABLE tags
  3. Build NetworkMessage (type = RPC_CALL, payload = RPCHeader + argData)
  |
  v
[C++ GameClient] SendToServer(message)
  1. Serialize NetworkMessage (10-byte header + payload)
  2. Check size vs MAX_PACKET_SIZE
     - Small: Pack into DATA packet, sendto()
```

```
     - Large: Fragment into FRAGMENT packets, send each
  |
  v
[Network] UDP datagrams travel to server
  |
  v
[C++ ReliabilityLayer I/O Thread] ReceivePacket()
  1. recvfrom() -> raw bytes
  2. Packet::Deserialize() -> validate checksum
  3. Send ACK back
  4. For FRAGMENT: reassemble, then treat as DATA
  5. NetworkMessage::Deserialize() -> extract header + payload
  6. Enqueue to incomingMessages
  |
  v
[C++ GameServer] Update()
  1. Dequeue message, switch on messageType
  2. RPC_CALL -> LuaRPCDeserializer::Deserialize()
     a. Read RPCMessageHeader (65 bytes)
     b. lua_getglobal(L, "ServerRPC_OnSubmitActions")
     c. For each arg: read type tag, read value, push to Lua stack
        - Tables/arrays reconstructed recursively
     d. lua_pcall(L, argCount, 0, 0)
  |
  v
[Lua Server] ServerRPC_OnSubmitActions(clientID, moveSupportQueue, attackQueue) executes
```

### 6.3 RPC Call Flow (Server to Specific Client)

```
[Lua Server] RPC.sendToClient(playerID, "ClientRPC_OnResolutionData", data)
  |
  v
[C++ GGLuaRPCBindings] lua_RPC_SendToClient(L)
  1. Extract playerID, function name, serialize args
  2. Build NetworkMessage
  |
  v
[C++ GameServer] SendToClient(message, playerID)
  1. Look up connection by PlayerID
  2. Serialize + fragment if needed
  3. sendto() to client's address
  |
  v
[Network] -> [ReliabilityLayer] -> [GameClient::Update()] -> [LuaRPCDeserializer]
  |
  v
[Lua Client] ClientRPC_OnResolutionData(data) executes
```

---

## 7. Wire Protocol Specification

### 7.1 Single DATA Packet (Small Message)

```
Byte 0-8:   PacketHeader (9 bytes)
              [sequenceNum: 4][payloadSize: 2][checksum: 2][packetType=0: 1]
Byte 9-18:  MessageHeader (10 bytes)
```

```
              [messageType: 2][payloadSize: 4][messageID: 4]
Byte 19-N:  Message Payload (variable)
              For RPC_CALL:
                [RPCMessageHeader: 65 bytes]
                [Argument data: variable]
```

### 7.2 FRAGMENT Packet

```
Byte 0-8:   PacketHeader (9 bytes)
              [sequenceNum: 4][payloadSize: 2][checksum: 2][packetType=5: 1]
Byte 9-16:  FragmentHeader (8 bytes)
              [messageID: 4][fragmentIndex: 2][fragmentTotal: 2]
Byte 17-N:  Fragment Data (up to MAX_FRAGMENT_PAYLOAD bytes)
```

### 7.3 ACK Packet

```
Byte 0-8:   PacketHeader (9 bytes)
              [sequenceNum: 4][payloadSize=0: 2][checksum: 2][packetType=1: 1]
```

No payload. The `sequenceNum` echoes the sequence number being acknowledged.

### 7.4 HEARTBEAT Packet

```
Byte 0-8:   PacketHeader (9 bytes)
              [sequenceNum: 4][payloadSize=0: 2][checksum: 2][packetType=2: 1]
```

---

## 8. Threading Model & Synchronization

### 8.1 Thread Architecture

```
+---------------------------+       +---------------------------+
|    Main/Game Thread       |       |    I/O Thread             |
|                           |       |    (ReliabilityLayer)     |
|  GameClient::Update()     |       |                           |
|  GameServer::Update()     |       |  ReceiveThreadProc():     |
|                           |       |    - recvfrom()           |
|  Reads coldMessages       |       |    - Deserialize packets  |
|  (after swap)             |       |    - Send ACKs            |
|                           |       |    - Fragment reassembly  |
|  Processes Lua callbacks  |       |    - Retransmission scan  |
|                           |       |    - Enqueue to           |
|  Calls sendto() directly  |       |      incomingMessages     |
|  (for outgoing messages)  |       |                           |
+---------------------------+       +---------------------------+
            |                                   |
            +--------- shared state ------------+
              connections (shared_mutex)
              incomingMessages (shared_mutex)
```

### 8.2 Synchronization Primitives

| Mutex | Type | Protects | Usage Pattern |
|---|---|---|---|

| | | | |
|---|---|---|---|
| `connectionsMutex` | `std::shared_mutex` | `connections` map | I/O thread: unique_lock for writes; Main thread: shared_lock for reads and sends |
| `messagesMutex` | `std::shared_mutex` | `incomingMessages` queue | I/O thread: unique_lock when enqueueing; Main thread: unique_lock during swap |

### 8.3 Double-Buffer Pattern (Client)

The client uses a double-buffer swap to minimize lock contention:

```
// GameClient::Update()
std::unique_lock lock(reliabilityLayer->messagesMutex);
std::swap(reliabilityLayer->incomingMessages, reliabilityLayer->coldMessages);
lock.unlock();

// Now process coldMessages without holding any lock
while (!reliabilityLayer->coldMessages.empty()) { ... }
```

The server processes `incomingMessages` directly under lock (simpler but holds the lock longer during processing).

### 8.4 Atomic Flags

- `shouldRun` ( `std::atomic<bool>` ) - Controls I/O thread lifecycle
- `shouldStopAtomic` ( `std::atomic<bool>` ) - Additional stop signal (unused in current implementation)

Both use `memory_order_release` / `memory_order_acquire` semantics for visibility guarantees.

---

## 9. RPC System Deep Dive

### 9.1 Supported Lua Types and Wire Encoding

| Lua Type | ArgType | Wire Encoding | Max Size |
|---|---|---|---|
| `nil` | `ARG_NIL` (0) | `[0x00]` | 1 byte |
| `boolean` | `ARG_BOOLEAN` (1) | `[0x01][0x00 or 0x01]` | 2 bytes |
| `integer` | `ARG_INTEGER` (2) | `[0x02][int32 LE]` | 5 bytes |
| `string` | `ARG_STRING` (3) | `[0x03][len: uint8][chars...]` | 257 bytes |
| `number` (float) | `ARG_NUMBER` (4) | `[0x04][double LE]` | 9 bytes |
| `lightuserdata` | `ARG_ENTITY` (5) | `[0x05][uint64 LE]` | 9 bytes |
| `table` (array) | `ARG_ARRAY` (6) | `[0x06][count: uint16 LE][elements...]` | Recursive |
| `table` (mixed) | `ARG_TABLE` (7) | `[0x07][count: uint16 LE][key,val,key,val...]` | Recursive |

### 9.2 Table Classification Algorithm

When serializing a Lua table ( `GGLuaRPCBindings.cpp:193` ):

1. Get sequence length via `lua_objlen()` (the `#` operator)
2. Count total entries by iterating with `lua_next()`
3. Classify:
   - **Array** if: `totalEntries == 0` (empty table) OR ( `seqLen > 0` AND `totalEntries == seqLen` )
   - **Table** otherwise (mixed keys, string keys, or gaps in indices)

### 9.3 Integer vs. Float Detection

Numbers are serialized as integers when: `floor(n) == n && n >= INT_MIN && n <= INT_MAX`

This allows Lua's unified `number` type to map efficiently to the narrower `int32_t` when the value is integral.

### 9.4 Example RPC Wire Format

```
RPC.call("ServerRPC_OnSubmitActions", 3, { {actionType=1, targetX=2, targetY=3} }, {})
```

Would produce:

```
[MessageHeader: messageType=199, payloadSize=..., messageID=...]
[RPCMessageHeader: nameLen=26, name="ServerRPC_OnSubmitActions\0...", argCount=3]
[ARG_INTEGER][03 00 00 00]                          -- clientID = 3
[ARG_ARRAY][01 00]                                   -- moveSupportQueue: 1 element
  [ARG_TABLE][03 00]                                 -- table with 3 entries
    [ARG_STRING][0A]"actionType" [ARG_INTEGER][01 00 00 00]
    [ARG_STRING][07]"targetX"   [ARG_INTEGER][02 00 00 00]
    [ARG_STRING][07]"targetY"   [ARG_INTEGER][03 00 00 00]
[ARG_ARRAY][00 00]                                   -- attackQueue: empty array
```

---

## 10. Turn-Based Game Flow

### 10.1 Complete Game Session Sequence

```
Phase 1: CONNECTION
  Client A connects, joins/creates lobby as Order
  Client B connects, joins lobby as Chaos

Phase 2: DEPLOYMENT
  Both clients place 3 units on their respective grids
  Each placement -> RPC to server -> broadcast to both
  Both click "End Deployment" -> server validates all units placed
  Server broadcasts all unit positions -> both clients transition to Prep

Phase 3: PREPARATION (repeats each turn)
  Timer starts (configurable seconds)
  Players queue movements (1 per unit, no AP cost) and abilities (costs AP)
  Actions stored locally in moveSupportQueue and attackQueue
  On submit -> RPC.call("ServerRPC_OnSubmitActions", clientID, moveQ, attackQ)
  Server waits for both players

Phase 4: RESOLUTION (repeats each turn)
  Server consolidates actions: Order moves -> Chaos moves -> Order attacks -> Chaos attacks
  Server pre-calculates all results:
    - Movement with trap checking (pathfinding)
    - Ability damage/healing with shield mechanics
    - Tile damage and destruction
    - Player health changes
  Server broadcasts ClientRPC_OnResolutionData to both clients
  Clients replay actions sequentially with animations:
    - Movement animations with trap hit pauses
    - Ability cast animations + VFX
    - Damage numbers, camera shakes
```

```
    - Health bar updates
  After all actions resolved:
    - Check for winner (health <= 0)
    - If no winner: return to Phase 3, increment turn number
    - If winner: display victory/defeat screen

Phase 5: GAME OVER
  Winner displayed, game phase set to "gameover"
  Server rejects further actions from this lobby
```

### 10.2 Action Queue Execution Order

The server processes queued actions in this deterministic order:

1. **Order Move/Support** - Movement commands + support abilities (shields, heals, traps)
2. **Chaos Move/Support** - Movement commands + support abilities
3. **Order Attack** - Offensive abilities
4. **Chaos Attack** - Offensive abilities

This ensures:

- Both players' movements resolve before any attacks
- Support abilities (shields, heals) apply before attacks
- Traps are placed and armed in predictable order
- The "first-mover advantage" of Order is balanced by ability design

### 10.3 Key Game Mechanics Over Network

**Shields**: Server tracks `shieldedUnits` map. Shield abilities add entries before attacks are processed. When a shielded unit is hit, the shield absorbs the hit (damage = 0, shield consumed).

**Traps**: Placed during resolution as "unarmed". Armed at the start of the next turn's resolution. Triggered when any unit moves through the trapped tile (checked via pathfinding along movement path).

**Tile Health**: Each grid tile has health ( `TILE_MAX_HEALTH` ). Abilities that miss units damage tiles instead. Destroyed tiles may affect gameplay.

**AP System**: Each player has `MAX_AP` per turn. Abilities cost AP (typically 1). Server validates total AP cost doesn't exceed limit.

---

## 11. Complete File Inventory

Every source file that implements the networking feature, organized by layer.

### 11.1 Layer 1: GGNetworking (Static Library)

| File | Path | Role |
|---|---|---|
| `PlatformWindows.hpp` | `GGNetworking/include/` | Winsock 2.2 init/shutdown declarations |
| `PlatformWindows.cpp` | `GGNetworking/src/backends/` | Winsock 2.2 `WSAStartup/WSACleanup` implementation |
| `UDPTypes.hpp` | `GGNetworking/include/` | Type aliases (`PlayerID`, `SessionID`, `SequenceNum`), enums (`PacketType`, `MessageType`), constants, `FragmentHeader` |
| `UDPPacket.hpp` | `GGNetworking/include/` | `PacketHeader` (9 bytes) and `Packet` class declarations |
| `UDPPacket.cpp` | `GGNetworking/src/network/` | Packet serialize/deserialize, RFC 1071 checksum computation |
| `UDPMessage.hpp` | `GGNetworking/include/` | `MessageHeader` (10 bytes), `NetworkMessage` class with `IsNetworkSerializable` concept |

| | | |
|---|---|---|
| UDPMessage.cpp | GGNetworking/src/network/ | NetworkMessage serialize/deserialize, WriteValue<T>/ReadValue<T> |
| UDP.hpp | GGNetworking/include/ | ClientConnection struct (per-client state: address, sequences, pending packets, fragment buffer) |
| UDP.cpp | GGNetworking/src/network/ | ClientConnection implementation |
| UDPReliabilityLayer.hpp | GGNetworking/include/ | ReliabilityLayer class: I/O thread, connection map, message queues, stats |
| UDPReliabilityLayer.cpp | GGNetworking/src/network/ | I/O thread main loop, ReceivePacket(), ACK handling, retransmission, fragment reassembly |

### 11.2 Layer 2: GGEngine (Engine Integration)

| File | Path | Role |
|---|---|---|
| GameClient.hpp | GGEngine/include/ | Client-side network manager: singleton, Init/Connect/Update/Stop, time sync |
| GameClient.cpp | GGEngine/src/Networking/ | Client socket setup, connection handshake, message processing loop, ping/pong, fragmentation |
| GameServer.hpp | GGEngine/include/ | Server-side network manager: singleton, Init/Start/Update/Stop, broadcast/send |
| GameServer.cpp | GGEngine/src/Networking/ | Server socket setup, message processing, heartbeat timeout, broadcast/targeted send, fragmentation |
| RPCMessage.hpp | GGEngine/include/ | RPCMessageHeader (65 bytes), ArgType enum, LuaRPCSerializer, LuaRPCDeserializer declarations |
| RPCMessage.cpp | GGEngine/src/Networking/ | RPC serialization/deserialization: argument packing, Lua stack push, lua_pcall execution |
| GGLuaRPCBindings.hpp | GGEngine/include/ | Lua RPC table function declarations (call, startServer, connectToServer, sendToClient) |
| GGLuaRPCBindings.cpp | GGEngine/src/ScriptingSystem/Networking/ | Lua binding registration, WriteLuaValueToRPC(), table/array classification, routing logic |
| GGGameCoordinator.hpp | GGEngine/include/ | Game coordinator: network state tracking, sync state machine, host/client mode |
| GGGameCoordinator.cpp | GGEngine/src/Engine_Core/System/ | Coordinator update: interrupt handling, connection-loss detection, game state callbacks |

### 11.3 Layer 3: m5_networking (Lua Gameplay Scripts)

| File | Path | Role |
|---|---|---|
| **System** | | |
| ConnectionCallbacks.lua | Assets/Scripts/m5_networking/System/ | Bridges C++ connection events to Lua local events (OnClientConnected, OnConnectedToServer, etc.) |
| SessionManager.lua | Assets/Scripts/m5_networking/System/ | Session tracking and persistence |

| | | |
|---|---|---|
| ArsenalData.lua | Assets/Scripts/m5_networking/System/ | Unit/ability data definitions shared between client and server |
| **Server** | | |
| NetworkedGameServer.lua | Assets/Scripts/m5_networking/Server/ | Authoritative server: lobby management, unit deployment validation, action validation, resolution algorithm |
| **Client - Core** | | |
| NetworkedGameClient.lua | Assets/Scripts/m5_networking/Client/ | Client connection manager: connect, lobby join, RPC callbacks for game events |
| ClientGameManager.lua | Assets/Scripts/m5_networking/Client/ | Client-side game state manager |
| **Client - Grid** | | |
| ClientGrid.lua | Assets/Scripts/m5_networking/Client/Grid/ | Client grid state representation |
| ClientGridLink.lua | Assets/Scripts/m5_networking/Client/Grid/ | Grid visual linking and tile interaction |
| **Client - Turn Phases** | | |
| ClientTurnPhases.lua | Assets/Scripts/m5_networking/Client/TurnState/ | Turn phase state machine controller |
| ClientDeploymentPhase.lua | Assets/Scripts/m5_networking/Client/TurnState/ | Deployment phase: unit placement logic |
| ClientPreparationPhase.lua | Assets/Scripts/m5_networking/Client/TurnState/ | Preparation phase: action queue, timer, submit |
| ClientResolutionPhase.lua | Assets/Scripts/m5_networking/Client/TurnState/ | Resolution phase: replay animations, apply results |
| **Client - Units** | | |
| ClientUnit.lua | Assets/Scripts/m5_networking/Client/Unit/ | Unit representation and state |
| ClientUnitStateMachine.lua | Assets/Scripts/m5_networking/Client/Unit/StateMachine/ | Unit animation state machine |
| ClientUnitStates.lua | Assets/Scripts/m5_networking/Client/Unit/StateMachine/States/ | State registry |
| ClientDeployState.lua | .../States/DeployState/ | Deploy animation state |
| ClientHitState.lua | .../States/HitState/ | Hit reaction animation state |
| ClientMovementStartupState.lua | .../States/MovementStartupState/ | Movement startup animation |
| ClientProjectedMoveState.lua | .../States/ProjectedMoveState/ | Projected movement preview |
| ClientReadyState.lua | .../States/ReadyState/ | Idle/ready state |
| ClientReplayMoveState.lua | .../States/ReplayMoveState/ | Resolution-phase movement replay |
| ClientSupportCastState.lua | .../States/SupportCastState/ | Support ability cast animation |

| **Client - Abilities** | | |
|---|---|---|
| `ClientAbilities.lua` | `Assets/Scripts/m5_networking/Client/Abilities/` | Ability collection manager |
| `ClientAbility.lua` | `Assets/Scripts/m5_networking/Client/Abilities/` | Base ability class |
| `ClientDragoonAbility1.lua` | `.../Abilities/Dragoon/` | Dragoon primary ability |
| `ClientDragoonAbility2.lua` | `.../Abilities/Dragoon/` | Dragoon secondary ability |
| `ClientEyeAbility1.lua` | `.../Abilities/Eye/` | Eye primary ability |
| `ClientEyeAbility2.lua` | `.../Abilities/Eye/` | Eye secondary ability |
| `ClientImpAbility1.lua` | `.../Abilities/Imp/` | Imp primary ability |
| `ClientImpAbility2.lua` | `.../Abilities/Imp/` | Imp secondary ability |
| `ClientOracleAbility1.lua` | `.../Abilities/Oracle/` | Oracle primary ability |
| `ClientOracleAbility2.lua` | `.../Abilities/Oracle/` | Oracle secondary ability |
| `ClientTirionAbility1.lua` | `.../Abilities/Tirion/` | Tirion primary ability |
| `ClientTirionAbility2.lua` | `.../Abilities/Tirion/` | Tirion secondary ability |
| `ClientVizomAbility1.lua` | `.../Abilities/Vizom/` | Vizom primary ability |
| `ClientVizomAbility2.lua` | `.../Abilities/Vizom/` | Vizom secondary ability |
| **UI** | | |
| `NetworkRPCHandler.lua` | `Assets/Scripts/m5_networking/` | Global RPC handler skeleton |
| `NetworkTestButton.lua` | `Assets/Scripts/m5_networking/` | Dev/test UI for starting server/client |
| `TestNetworkingLobby.lua` | `Assets/Scripts/m5_networking/UI/` | Lobby test scene |
| `TestLobbyScene.lua` | `Assets/Scripts/m5_networking/UI/` | Lobby scene test |
| `TestModeSelection.lua` | `Assets/Scripts/m5_networking/UI/` | Mode selection test |
| `TestCompleteFlow.lua` | `Assets/Scripts/m5_networking/UI/` | Full flow test |
| `TestArsenal.lua` | `Assets/Scripts/m5_networking/UI/` | Arsenal test scene |
| `UI_LobbySelection.lua` | `Assets/Scripts/m5_networking/UI/` | Lobby selection screen |
| `UI_LobbyContainer.lua` | `Assets/Scripts/m5_networking/UI/` | Lobby container layout |
| `UI_LobbyRoom.lua` | `Assets/Scripts/m5_networking/UI/` | Lobby room display |
| `UI_RoomEntry.lua` | `Assets/Scripts/m5_networking/UI/` | Room entry widget |
| `UI_CreateRoomButton.lua` | `Assets/Scripts/m5_networking/UI/` | Create room button |
| `UI_JoinRoom.lua` | `Assets/Scripts/m5_networking/UI/` | Join room button |
| `UI_LeaveRoomButton.lua` | `Assets/Scripts/m5_networking/UI/` | Leave room button |
| `UI_RefreshRoomsButton.lua` | `Assets/Scripts/m5_networking/UI/` | Refresh rooms button |
| `UI_LobbyBackButton.lua` | `Assets/Scripts/m5_networking/UI/` | Lobby back button |

| | | |
|---|---|---|
| UI_LobbyArsenalButton.lua | Assets/Scripts/m5_networking/UI/ | Arsenal access from lobby |
| UI_LobbyPlayerDisplay.lua | Assets/Scripts/m5_networking/UI/ | Player display in lobby |
| UI_ReadyButton.lua | Assets/Scripts/m5_networking/UI/ | Ready up button |
| UI_PlayButton.lua | Assets/Scripts/m5_networking/UI/ | Start game button |
| UI_ModeSelection.lua | Assets/Scripts/m5_networking/UI/ | Online/Pass-and-Play mode selection |
| UI_ModeSelectionBackButton.lua | Assets/Scripts/m5_networking/UI/ | Mode selection back button |
| UI_OnlineButton.lua | Assets/Scripts/m5_networking/UI/ | Online mode button |
| UI_PassAndPlayButton.lua | Assets/Scripts/m5_networking/UI/ | Pass-and-play mode button |
| UI_PlayAsChaosButton.lua | Assets/Scripts/m5_networking/UI/ | Play as Chaos faction |
| UI_PlayAsOrderButton.lua | Assets/Scripts/m5_networking/UI/ | Play as Order faction |
| UI_FactionSelection.lua | Assets/Scripts/m5_networking/UI/ | Faction selection screen |
| UI_FactionIconDisplay.lua | Assets/Scripts/m5_networking/UI/ | Faction icon renderer |
| UI_FactionLeftArrow.lua | Assets/Scripts/m5_networking/UI/ | Faction selection left arrow |
| UI_FactionRightArrow.lua | Assets/Scripts/m5_networking/UI/ | Faction selection right arrow |
| UI_Arsenal.lua | Assets/Scripts/m5_networking/UI/ | Arsenal screen |
| UI_ArsenalButton.lua | Assets/Scripts/m5_networking/UI/ | Arsenal button |
| UI_ArsenalBackButton.lua | Assets/Scripts/m5_networking/UI/ | Arsenal back button |
| UI_ArsenalConfirm.lua | Assets/Scripts/m5_networking/UI/ | Arsenal confirm button |
| UI_ArsenalUnit.lua | Assets/Scripts/m5_networking/UI/ | Arsenal unit display |
| UI_ArsenalAbilityIcon.lua | Assets/Scripts/m5_networking/UI/ | Arsenal ability icon |
| UI_ArsenalAbilitySlot.lua | Assets/Scripts/m5_networking/UI/ | Arsenal ability slot |
| UI_ArsenalDragDrop.lua | Assets/Scripts/m5_networking/UI/ | Arsenal drag-and-drop handler |
| UI_ArsenalUITab.lua | Assets/Scripts/m5_networking/UI/ | Arsenal tab navigation |
| UI_OrderPlayerDisplay.lua | Assets/Scripts/m5_networking/UI/ | Order player in-game display |
| UI_ChaosPlayerDisplay.lua | Assets/Scripts/m5_networking/UI/ | Chaos player in-game display |

### 11.4 Entry Point

| File | Path | Role |
|---|---|---|
| NetworkManager.lua | Assets/Scripts/ | Scene-level entry point: detects server vs. client mode from serializable fields, calls RPC.startServer() or RPC.connectToServer() |

### 11.5 Engine Integration Point

| File | Path | Role |
|---|---|---|

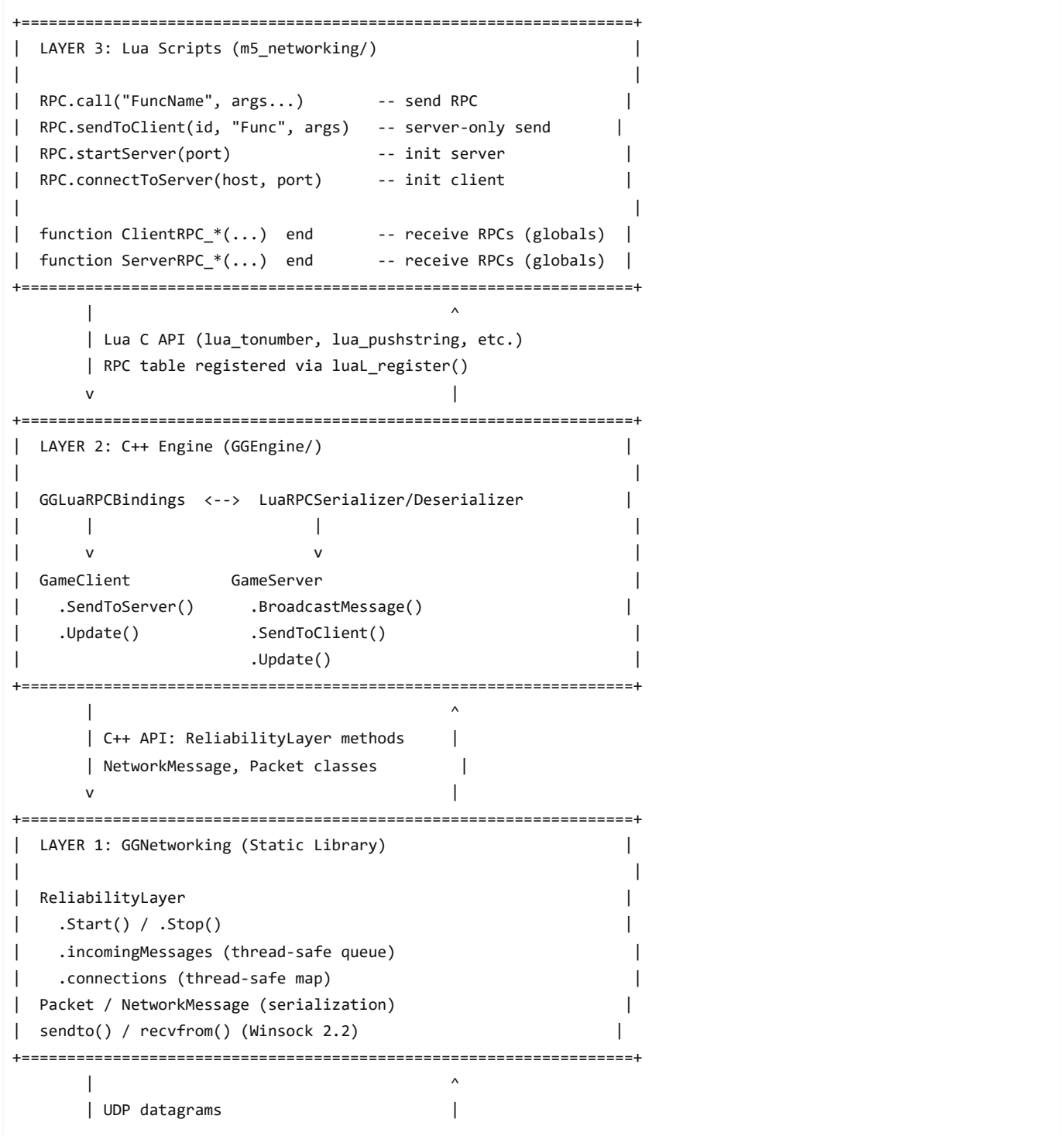
| | | |
|---|---|---|
| GGEngine.cpp | GGEngine/src/Engine_Core/ | Main engine update loop; calls `GameServer::Update()` and `GameClient::Update()` |
| GGLuaBindings.cpp | GGEngine/src/ScriptingSystem/Lua_Core/ | Registers all Lua bindings including RPC bindings via `RegisterAllLuaBindings()` |

---

## 12. Inter-Layer Communication

This section describes how the three networking layers communicate with each other, covering both the interfaces and the data that flows across each boundary.

### 12.1 Layer Boundary Diagram



```
        v                                       |
+=================================================================+
|  OS Network Stack (Winsock 2.2 / WS2_32.lib)                    |
+=================================================================+
```

### 12.2 Layer 3 -> Layer 2 (Lua calls C++)

**Interface**: The `RPC` global Lua table, registered via `GGLuaRPCBindings::RegisterLuaRPCBindings()`.

**How it works**:

1. Lua script calls `RPC.call("ServerRPC_OnSubmitActions", clientID, moveQueue, attackQueue)`
2. The Lua C API invokes the C function `lua_RPC_Call(lua_State* L)` in `GGLuaRPCBindings.cpp`
3. C++ reads arguments from the Lua stack using `lua_tostring()`, `lua_tonumber()`, `lua_istable()`, etc.
4. `WriteLuaValueToRPC()` recursively serializes each Lua value into binary format using `LuaRPCSerializer`
5. The serializer produces a `NetworkMessage` with `MessageType::RPC_CALL`
6. Routing decision:
    - If `GameClient` is running: `GameClient::SendToServer(message)`
    - If `GameServer` is running: `GameServer::BroadcastMessage(message)`

**Data crossing this boundary**: Lua values (booleans, numbers, strings, tables, entities) are converted to tagged binary format (`ArgType` + raw bytes).

### 12.3 Layer 2 -> Layer 3 (C++ calls Lua)

**Interface**: `GGEngine::CallLuaFunctionAdvanced(functionName, args)` which performs `lua_getglobal()` + `lua_pcall()`.

**How it works**:

1. `GameClient::Update()` or `GameServer::Update()` dequeues an `RPC_CALL` message
2. `LuaRPCDeserializer::Deserialize()` reads the `RPCMessageHeader` to get the function name
3. For each argument, it reads the `ArgType` tag and value bytes, then pushes the corresponding Lua value onto the stack
4. Calls `lua_pcall(L, argCount, 0, 0)` to execute the Lua function

**Data crossing this boundary**: Binary `ArgType`-tagged bytes are converted back to Lua values (`lua_pushboolean`, `lua_pushinteger`, `lua_pushstring`, `lua_createtable`, etc.).

**Connection events** also cross this boundary directly (not via RPC):

- `GameClient::Update()` calls `CallLuaFunctionAdvanced("ClientRPC_OnConnectedToServer", {playerID})` on successful handshake
- `GameServer::Update()` calls `CallLuaFunctionAdvanced("ServerRPC_OnClientConnected", {playerID})` on new connection
- Disconnect callbacks follow the same pattern

### 12.4 Layer 2 -> Layer 1 (Engine uses Transport)

**Interface**: `ReliabilityLayer` public API + direct `sendto()` calls.

**How it works**:

1. `GameClient` / `GameServer` create a `ReliabilityLayer` in their `Init()`, passing the bound UDP socket
2. They call `ReliabilityLayer::Start()` to spawn the I/O thread
3. For sending: `GameClient::SendToServer()` / `GameServer::SendToClient()` serialize the `NetworkMessage`, wrap it in a `Packet` (with `PacketHeader`), and call `sendto()` directly on the socket. Large messages are fragmented into multiple `FRAGMENT` packets before sending.
4. For receiving: The main thread reads `ReliabilityLayer::incomingMessages` (or `coldMessages` after swap) which are populated by the I/O thread.

**Data crossing this boundary**:

- **Outbound**: `NetworkMessage` -> `Packet` serialized bytes -> `sendto()` UDP datagram
- **Inbound**: UDP datagram -> `Packet::Deserialize()` -> `NetworkMessage::Deserialize()` -> enqueued as `pair<NetworkMessage, PlayerID>`

### 12.5 Layer 1 -> Network (Transport to Wire)

**Interface**: Winsock 2.2 `sendto()` / `recvfrom()` via non-blocking UDP sockets.

**How it works**:

- Non-blocking sockets ( `ioctlsocket(FIONBIO)` ) ensure no thread blocks on I/O
- The I/O thread continuously polls `recvfrom()` with a 1ms sleep when no data available
- Outgoing packets are sent via `sendto()` from the main thread (for GameClient/GameServer sends) or I/O thread (for ACKs)

**Data crossing this boundary**: Raw bytes in UDP datagrams. Maximum datagram size is `MAX_PACKET_SIZE` (1400 bytes) to stay under typical MTU.

### 12.6 Cross-Layer Call Sequence (Complete Example)

A client submitting actions, end-to-end:

```
SENDER SIDE:
  [Lua]   RPC.call("ServerRPC_OnSubmitActions", 3, moveQ, attackQ)
     |
     | --- Layer 3->2 boundary (Lua C API) ---
     v
  [C++]   lua_RPC_Call(L)
          WriteLuaValueToRPC(L, 2, serializer)  -- serialize arg 1 (int)
          WriteLuaValueToRPC(L, 3, serializer)  -- serialize arg 2 (table)
          WriteLuaValueToRPC(L, 4, serializer)  -- serialize arg 3 (table)
          serializer.Serialize(networkMessage)   -- produce NetworkMessage
     |
     | --- Layer 2->1 boundary (C++ API) ---
     v
  [C++]   GameClient::SendToServer(networkMessage)
          networkMessage.Serialize() -> bytes
          Packet::Serialize(header + bytes)
          sendto(socket, packetBytes, serverAddr)
     |
     | --- Layer 1->Network boundary (Winsock) ---
     v
  [Wire]  UDP datagram travels to server

RECEIVER SIDE:
  [Wire]  UDP datagram arrives at server socket
     |
     | --- Network->Layer 1 boundary (Winsock) ---
     v
  [C++]   ReliabilityLayer::ReceivePacket()     [I/O thread]
          recvfrom() -> raw bytes
          Packet::Deserialize() -> validate checksum
          NetworkMessage::Deserialize() -> extract header + payload
          Send ACK back
          Enqueue to incomingMessages
     |
     | --- Layer 1->2 boundary (thread-safe queue) ---
     v
  [C++]   GameServer::Update()                  [Main thread]
```

```
          Dequeue from incomingMessages
          switch(RPC_CALL) -> LuaRPCDeserializer::Deserialize()
          Read RPCMessageHeader: functionName = "ServerRPC_OnSubmitActions"
          ReadAndPushArg() x3 -> push values to Lua stack
      |
      | --- Layer 2->3 boundary (Lua C API) ---
      v
  [Lua]   ServerRPC_OnSubmitActions(3, moveQ, attackQ)  executes
```

---

## 13. GameClient & GameServer in the Engine Update Loop

This section documents exactly where network updates occur within the `GGEngine::Update()` main loop, based on `GGEngine.cpp` lines 537-723.

### 13.1 Full Update Order

```
GGEngine::Update()
|
|-- while (!glfwWindowShouldClose(window))
    |
    |-- [1] GGInterruptHandler::Update()          // Check ALT-TAB, minimize, focus loss
    |
    |-- [2] if (IsSystemPaused) {                  // OS INTERRUPT PATH
    |       glfwPollEvents()
    |       glfwSwapBuffers()
    |       GameServer::Update()   <-- line 563    // KEEP HEARTBEATS ALIVE
    |       GameClient::Update()   <-- line 567    // KEEP HEARTBEATS ALIVE
    |       continue;                              // SKIP all game logic
    |   }
    |
    |-- [3] TimeManager::Update()                  // Delta time calculation
    |-- [4] PerformanceProfiler::BeginFrame()      // Profiling start
    |-- [5] glfwPollEvents()                       // Input: keyboard, mouse, window
    |
    |-- [6] ECSScript::Update()                    // ** Lua scripts execute **
    |-- [7] ECSTransform::Update()                 // Transform hierarchy
    |-- [8] ECSHierarchy::Update()                 // Parent-child relationships
    |-- [9] SimulationManager::Update()            // Physics (fixed timestep)
    |-- [10] TimerManager::Update()                // Timer callbacks
    |-- [11] ECSAudio::Update()                    // Audio system
    |
    |-- [12] GameServer::Update()  <-- line 599    // ** PROCESS NETWORK MESSAGES **
    |-- [13] GameClient::Update()  <-- line 604    // ** PROCESS NETWORK MESSAGES **
    |
    |-- [14] ProcessDeferredDestruction()           // Destroy entities queued by RPCs
    |
    |-- [15] Picking framebuffer render             // Mouse picking
    |-- [16] ECSRender::RenderPicking()
    |-- [17] ECSRender::RenderUIPicking()
    |-- [18] Audio system update
    |-- [19] ECSRender::Update()                   // Main scene render
    |-- [20] ImGui editor overlay
    |-- [21] glfwSwapBuffers()                     // Present frame
    |-- [22] Frame limiter (busy-wait for 60 FPS)
```

### 13.2 Why This Ordering Matters

**Network updates happen AFTER scripting but BEFORE rendering (steps 12-13):**

1. **Scripts execute first (step 6)**: Lua `Update()` functions run, which may call `RPC.call()`. These RPCs serialize and send immediately via `sendto()`. However, any *incoming* messages from the previous frame's network activity are not yet processed at this point.
2. **Network updates process incoming messages (steps 12-13)**: `GameServer::Update()` and `GameClient::Update()` dequeue messages that the I/O thread received since the last frame. This includes:
   - RPC calls from remote peers -> trigger Lua function execution (e.g., `ClientRPC_OnResolutionData`)
   - Connection/disconnection events -> trigger Lua callbacks
   - Ping/pong for time synchronization
3. **Deferred destruction follows immediately (step 14)**: Any entities destroyed by RPC handlers are cleaned up before rendering, preventing stale entity rendering.
4. **Rendering occurs last (steps 15-21)**: The visual state reflects all network updates from this frame.

**The I/O thread runs independently**: While the main thread follows this sequential order, the `ReliabilityLayer` I/O thread continuously receives UDP packets, sends ACKs, handles retransmission, and reassembles fragments. Messages arrive in the `incomingMessages` queue asynchronously between main-thread Update() calls.

### 13.3 OS Interrupt Handling (steps 1-2)

When the system is paused (ALT-TAB, window minimize), the engine **still updates networking**:

```
if (GGInterruptHandler::GetInstance()->IsSystemPaused()) {
    glfwPollEvents();
    glfwSwapBuffers(window);

    // Critical: maintain network connections during OS interrupts
    if (GGNetworking::GetGameServer()->IsRunning())
        GGNetworking::GetGameServer()->Update();   // Process pings, send pongs
    if (GGNetworking::GetGameClient()->IsRunning())
        GGNetworking::GetGameClient()->Update();   // Send pings, process pongs

    continue;  // Skip ALL game logic
}
```

This ensures:

- **Heartbeat packets continue**: The client sends pings every 2 seconds; the server responds with pongs. Without this, a 10-second ALT-TAB would trigger the server's `HEARTBEAT_TIMEOUT_US` and disconnect the client.
- **No game state changes**: Only networking housekeeping runs. No scripts, physics, or rendering execute during the interrupt.
- **Connection stability**: Players can safely ALT-TAB without disconnecting.

### 13.4 GameClient::Update() Internal Flow

Called at step 13 (or step 2 during OS interrupt). Source: `GameClient.cpp:238`.

```
GameClient::Update()
|
|-- Send CLIENT_PING if interval elapsed (2 seconds)
|
|-- Lock messagesMutex
|-- swap(incomingMessages, coldMessages)     // Double-buffer swap
```

```
|-- Unlock messagesMutex
|
|-- while (!coldMessages.empty())
    |-- Dequeue {NetworkMessage, PlayerID}
    |-- switch (messageType):
        |
        |-- SERVER_CONNECT_RESPONSE:
        |   Read PlayerID, server time
        |   Calculate clientServerOffset
        |   CallLuaFunctionAdvanced("ClientRPC_OnConnectedToServer", playerID)
        |
        |-- CLIENT_DISCONNECT_RESPONSE:
        |   Reset isConnected, isRunning
        |   CallLuaFunctionAdvanced("ClientRPC_OnDisconnectedFromServer")
        |
        |-- SERVER_PONG:
        |   Calculate RTT = now - lastPingSentTime
        |   Update clientServerOffset with half-RTT correction
        |
        |-- RPC_CALL:
        |   LuaRPCDeserializer::Deserialize(message)
        |   -> Reads function name from RPCMessageHeader
        |   -> Pushes arguments to Lua stack
        |   -> lua_pcall() executes the Lua function
```

### 13.5 GameServer::Update() Internal Flow

Called at step 12 (or step 2 during OS interrupt). Source: `GameServer.cpp:209` .

```
GameServer::Update()
|
|-- Lock messagesMutex
|-- while (!incomingMessages.empty())        // Direct read (no double-buffer)
    |-- Dequeue {NetworkMessage, senderPlayerID}
    |-- switch (messageType):
        |
        |-- CLIENT_CONNECT_REQUEST:
        |   Update connection timestamps
        |   CallLuaFunctionAdvanced("ServerRPC_OnClientConnected", playerID)
        |   Build SERVER_CONNECT_RESPONSE (PlayerID, server time, status)
        |   SendToClient(response, playerID)
        |
        |-- CLIENT_DISCONNECT_REQUEST:
        |   Remove from connections map
        |   CallLuaFunctionAdvanced("ServerRPC_OnClientDisconnected", playerID)
        |   Send CLIENT_DISCONNECT_RESPONSE
        |
        |-- CLIENT_PING:
        |   Update lastHeartbeat timestamp
        |   Build SERVER_PONG with server time
        |   SendToClient(pong, senderPlayerID)
        |
        |-- CLIENT_TURN_ACTION:
        |   BroadcastMessage(message, excludeSender)
        |
        |-- RPC_CALL:
```

```
        |   LuaRPCDeserializer::Deserialize(message)
        |   -> Execute Lua server RPC function
|
|-- Unlock messagesMutex
|
|-- Heartbeat timeout scan:
    |-- For each connection in ReliabilityLayer::connections:
        |-- if (now - lastHeartbeat > HEARTBEAT_TIMEOUT_US)
            |-- Mark connection as timed out
            |-- CallLuaFunctionAdvanced("ServerRPC_OnClientDisconnected", playerID)
    |-- Remove timed-out connections
```

### 13.6 Frame Timing and Determinism

The engine enforces a 60 FPS frame rate using a busy-wait loop after all updates:

```
auto frameEndTime = std::chrono::high_resolution_clock::now();
double targetFrameTime = 1.0 / 60.0;  // ~16.667ms

while (elapsed < targetFrameTime) {
    elapsed = (now - frameStartTime).count();
}
```

This ensures:

- Network messages are processed at a consistent rate (60 times per second)
- Turn-based resolution animations play at predictable speed
- Client ping measurements have consistent baseline timing

---

*Document generated from codebase analysis of GGPals Custom Engine, branch `GGPals-m5-network`. GGNetworking Library authored by Tai Xiao Xuan, DigiPen Institute of Technology, 2025.*